# Dokumentasi Resmi & Sintaks Bahasa C (C17 / C23 Up-to-Date)

Bahasa: C | Total Chunk/Dokumen: 45 | Disiapkan untuk RAG System

**[c_001] C - C Standards & Conformance (C11, C17, C23)**

## C Language Standards Conformance (C11, C17, and C23 Compiler Flags /std:c11 /std:c17 -std=c23): Overview

*Sumber: ht*

# Install C11 and C17 support in Visual Studio
::: moniker range="=msvc-160"
Support for C11 and C17 standards is available in Visual Studio 2019 version 16.8 and later. Support requires an updated Universal C Runtime (UCRT) and Windows SDK version to work properly with the conforming preprocessor (`/Zc:preprocessor`).
Windows SDK releases correspond with Windows OS releases. C11 and C17 support requires Windows SDK 10.0.20348.0 (version 2104) or later. The Windows SDK is an installable option in the **Individual Components** tab in the Visual Studio Installer. You can follow these steps to set up the latest SDK and build C11 or C17 code in Visual Studio or at the command line.

**[c_002] C - C Standards & Conformance (C11, C17, C23)**

## C Language Standards Conformance (C11, C17, and C23 Compiler Flags /std:c11 /std:c17 -std=c23): Prerequisites

*Sumber: ht*

## Prerequisites
Visual Studio 2019 version 16.8 or later and Windows SDK 10.0.20348.0 (version 2104) or later installed and running on your computer. We recommend you use the latest version available for the best support.
- If Visual Studio isn't installed yet, see Install C++ support in Visual Studio for installation instructions. In the install, include the **Desktop development with C++** workload. Then, open the Individual Components tab. Select Windows 10 SDK (10.0.20348.0) or later, or the latest Windows 11 SDK.
- If Visual Studio is already installed, but you don't have the required Windows SDK, open the Visual Studio Installer. Choose **Modify** next to your version of Visual Studio to install the selected components. Open the Individual Components tab. Select Windows 10 SDK (10.0.20348.0) or later, or the latest Windows 11 SDK. Choose **Modify** to install the selected components.
When installation completes, open Visual Studio.

**[c_003] C - C Standards & Conformance (C11, C17, C23)**

## C Language Standards Conformance (C11, C17, and C23 Compiler Flags /std:c11 /std:c17 -std=c23): Configure C11 or C17 mode in Visual Studio

*Sumber: ht*

## Configure C11 or C17 mode in Visual Studio
In Visual Studio, open a new or existing C project, then open your project's **Property Pages** dialog.
Set the project to use the Windows SDK you just installed. On the **Configuration Properties** > **General** page, set the **Windows SDK Version** property either to **10.0 (latest installed version)**, or to the specific SDK version you installed.
You'll also see a C-specific option: **C Language Standard**. Set this property to **ISO C11 Standard (`/std:c11`)** or **ISO C17 (2018) Standard (`/std:c17`)**.
!A screenshot of the Property Pages dialog on the Configuration Properties General page, showing the C Language Standard property drop-down selection as ISO C 17.
The C++ Language Standard property is used when the language is C++. It's the default when the file extension is

*`.cpp`*. The C Language Standard property version is used when the language is C. It's the default when the file extension is *`.c`*. To build using C11 or C17, put your source code in a *`.c`* file, or set the code to compile as C. You can set this property for your project on the **Configuration Properties** > **C/C++** > **Advanced** page. Set the **Compile As** property to **Compile as C code (/TC)**.
Congratulations, you've set up everything you need to build C11 and C17 code in Visual Studio!

**[c_004] C - C Standards & Conformance (C11, C17, C23)**

## C Language Standards Conformance (C11, C17, and C23 Compiler Flags /std:c11 /std:c17 -std=c23): See also

*Sumber: ht*

## See also
`/std` (Specify language standard version)
::: moniker-end

**[c_005] C - Data Types & Type Specifiers**

## C Type Specifiers & Data Types (int, float, double, char, _Bool, <stdint.h> fixed-width integers): Overview

*Sumber: ht*

# C Type Specifiers
Type specifiers in declarations define the type of a variable or function declaration.

**[c_006] C - Data Types & Type Specifiers**

## C Type Specifiers & Data Types (int, float, double, char, _Bool, <stdint.h> fixed-width integers): Syntax

*Sumber: ht*

## Syntax
*`type-specifier`*:
 **`void`**
 **`char`**
 **`short`**
 **`int`**
 **`long`**
 **`float`**
 **`double`**
 **`signed`**
 **`unsigned`**
 *`struct-or-union-specifier`*
 *`enum-specifier`*
 *`typedef-name`*
The **`signed char`**, **`signed int`**, **`signed short int`**, and **signed long int** types, together with their **`unsigned`** counterparts and **`enum`**, are called *integral* types. The **`float`**, **`double`**, and **`long double`** type specifiers are referred to as *floating* or *floating-point* types. You can use any integral or floating-point type specifier in a variable or function declaration. Originally, if a *`type-specifier`* wasn't provided in a declaration, it was taken to be **`int`**. The Microsoft compiler no longer accepts default **`int`** declarations.
The optional keywords **`signed`** and **`unsigned`** can precede or follow any of the integral types, except **`enum`**, and can also be used alone as type specifiers, in which case they're understood as **`signed int`** and **`unsigned int`**, respectively. When used alone, the keyword **`int`** is assumed to be **`signed`**. When used alone, the keywords **`long`** and **`short`** are understood as **long int** and **`short int`**.
Enumeration types are considered basic types. Type specifiers for enumeration types are discussed in Enumeration

Declarations.
The keyword **`void`** has three uses: to specify a function return type, to specify an argument-type list for a function that takes no arguments, and to specify a pointer to an unspecified type. You can use the **`void`** type to declare functions that return no value or to declare a pointer to an unspecified type. See Arguments for information on **`void`** when it app

**Contoh Kode / Perintah CLI:**

```
int   myfunc();
short myfunc();

// ----------------------------------------

int *pi;
short *ps;

ps = pi;  /* Now generates warning */

// ----------------------------------------

signed int *pi;
unsigned int *pu;

pi = pu;  /* Now generates warning */
```

**[c_007] C - Data Types & Type Specifiers**

## C Type Specifiers & Data Types (int, float, double, char, _Bool, <stdint.h> fixed-width integers): See also

*Sumber: ht*

## See also
Declarations and Types\
`typeof, __typeof__` (C23)\
`typeof_unqual, __typeof_unqual__` (C23)

**[c_008] C - Pointers & Memory Addressing**

## Pointer Declarations and Indirection (Pointer arithmetic, NULL, void*, double pointers **): Overview

*Sumber: ht*

# Pointer Declarations
A *pointer declaration* names a pointer variable and specifies the type of the object to which the variable points. A variable declared as a pointer holds a memory address.

**[c_009] C - Pointers & Memory Addressing**

## Pointer Declarations and Indirection (Pointer arithmetic, NULL, void*, double pointers **): Syntax

*Sumber: ht*

## Syntax
*`declarator`*:\
 *`pointer`*opt *`direct-declarator`*
*`direct-declarator`*:\
 *`identifier`*\
 **`(`** *`declarator`* **`)`**\
 *`direct-declarator`* **`[`** *`constant-expression`*opt **`]`**\

 *`direct-declarator`* **`(`** *`parameter-type-list`* **`)`**\
 *`direct-declarator`* **`(`** *`identifier-list`*opt **`)`**
*`pointer`*:\
 **`*`** *`type-qualifier-list`*opt\
 **`*`** *`type-qualifier-list`*opt *`pointer`*
*`type-qualifier-list`*:\
 *`type-qualifier`*\
 *`type-qualifier-list`* *`type-qualifier`*
The *`type-specifier`* gives the type of the object, which can be any basic, structure, or union type. Pointer variables can also point to functions, arrays, and other pointers. (For information on declaring and interpreting more complex pointer types, refer to Interpreting More Complex Declarators.)
By making the *`type-specifier`* **`void`**, you can delay specification of the type to which the pointer refers. Such an item is referred to as a "pointer to **`void`**" and is written as `void *`. A variable declared as a pointer to **`void`** can be used to point to an object of any type. However, to perform most operations on the pointer or on the object to which it points, the type to which it points must be explicitly specified for each operation. (Variables of type `char *` and type `void *` are assignment-compatible without a type cast.) Such conversion can be accomplished with a type cast. For more information, see Type-Cast Conversions.
The *`type-qualifier`* can be either **`const`** or **`volatile`**, or both. These keywords specify, respectively, that the pointer can't be modified by the program itself (**`const`**), or that the pointer can legitimately be modified by some process beyond the control

**[c_010] C - Pointers & Memory Addressing**

## Pointer Declarations and Indirection (Pointer arithmetic, NULL, void*, double pointers **): Examples

*Sumber: ht*

## Examples
The following examples illustrate pointer declarations.
The `message` pointer points to a variable with **`char`** type.
The `pointers` array has 10 elements; each element is a pointer to a variable with **`int`** type.
The `pointer` variable points to an array with 10 elements. Each element in this array has **`int`** type.
The pointer `x` can be modified to point to a different **`int`** value, but the value to which it points can't be modified.
The variable `y` in these declarations is declared as a constant pointer to an **`int`** value. The value it points to can be modified, but the pointer itself must always point to the same location: the address of `fixed_object`. Similarly, `z` is a constant pointer, but it's also declared to point to an **`int`** whose value can't be modified by the program. The **`volatile`** specifier indicates that although the value of the `const int` pointed to by `z` can't be modified by the program, it could legitimately be modified by a process running concurrently with the program. The declaration of `w` specifies that the program can't change the value pointed to and that the program can't modify the pointer.
This example declares two pointer variables (`next` and `previous`) that point to the structure type `list`. This declaration can appear before the definition of the `list` structure type (see the next example), as long as the `list` type definition has the same visibility as the declaration.
The variable `line` has the structure type named `list`. The `list` structure type has three members: the first member is a pointer to a **`char`** value, the second is an **`int`** value, and the third is a pointer to another `list` structure.
The variable `record` has the structure type `id`. `pname` is declared as a pointer

**Contoh Kode / Perintah CLI:**

```
char *message; /* Declares a pointer variable named message */

// ----------------------------------------

int *pointers[10];  /* Declares an array of pointers */

// ----------------------------------------
```

```
int (*pointer)[10]; /* Declares a pointer to an array of 10 elements */
```

[c_011] C - Pointers & Memory Addressing

## Pointer Declarations and Indirection (Pointer arithmetic, NULL, void*, double pointers **): See also

Sumber: ht

## See also
Declarators and Variable Declarations

[c_012] C - Structures & Designated Initializers

## Structure Declarations (struct, typedef struct, Member access, Designated Initializers .x = 1): Overview

Sumber: ht

# Structure Declarations
A "structure declaration" names a type and specifies a sequence of variable values (called "members" or "fields" of the structure) that can have different types. An optional identifier, called a "tag," gives the name of the structure type and can be used in subsequent references to the structure type. A variable of that structure type holds the entire sequence defined by that type. Structures in C are similar to the types known as "records" in other languages.

[c_013] C - Structures & Designated Initializers

## Structure Declarations (struct, typedef struct, Member access, Designated Initializers .x = 1): Syntax

Sumber: ht

## Syntax
*`struct-or-union-specifier`*:\
 *`struct-or-union`* *`identifier`*opt **`{`** *`struct-declaration-list`* **`}`**\
 *`struct-or-union`* *`identifier`*
*`struct-or-union`*:\
 **`struct`**\
 **`union`**
*`struct-declaration-list`*:\
 *`struct-declaration`*\
 *`struct-declaration-list`* *`struct-declaration`*
*`struct-declaration`*:\
 *`specifier-qualifier-list`* *`struct-declarator-list`* **`;`**
*`specifier-qualifier-list`*:\
 *`type-specifier`* *`specifier-qualifier-list`*opt\
 *`type-qualifier`* *`specifier-qualifier-list`*opt
*`struct-declarator-list`*:\
 *`struct-declarator`* *`struct-declarator-list`* **`,`** *`struct-declarator`*
*`struct-declarator`*:\
 *`declarator`*\
 *`type-specifier`* *`declarator`*opt **`:`** *`constant-expression`*
The declaration of a structure type doesn't set aside space for a structure. It's only a template for later declarations of structure variables.
A previously defined *`identifier`* (tag) can be used to refer to a structure type defined elsewhere. In this case, *`struct-declaration-list`* can't be repeated as long as the definition is visible. Declarations of pointers to structures and typedefs for structure types can use the structure tag before the structure type is defined. However, the structure definition must be encountered prior to any actual use of the size of the fields. This use is an incomplete definition of the

type and the type tag. For this definition to be completed, a type definition must appear later in the same scope.
The *`struct-declaration-list`* specifies the types and names of the structure members. A *`struct-declaration-list`* argument contains one or more variable or bit-field declarations.
Each variable declared in *`struct-declaration

**Contoh Kode / Perintah CLI:**

```
struct a
{
    int x;
    struct b
    {
      int y;
    } var2;
} var1;

// ----------------------------------------

struct a var3;
struct b var4;
```

**[c_014] C - Structures & Designated Initializers**

## Structure Declarations (struct, typedef struct, Member access, Designated Initializers .x = 1): Examples

*Sumber: ht*

## Examples
These examples illustrate structure declarations:
The `employee` structure has three members: `name`, `id`, and `class`. The `name` member is a 20-element array, and `id` and `class` are simple members with **`int`** and **`long`** type, respectively. The identifier `employee` is the structure identifier.
This example defines three structure variables: `student`, `faculty`, and `staff`. Each structure has the same list of three members. The members are declared to have the structure type `employee`, defined in the previous example.
The `complex` structure has two members with **`float`** type, `x` and `y`. The structure type has no tag and is therefore unnamed or anonymous.
The first two members of the structure are a **`char`** variable and a pointer to a **`float`** value. The third member, `next`, is declared as a pointer to the structure type being defined (`sample`).
Anonymous structures can be useful when the tag name isn't needed, such as when one declaration defines all structure instances. For example:
Embedded structures are often anonymous.
**Microsoft Specific**
The compiler allows an unsized or zero-sized array as the last member of a structure. It's useful if the size of a constant array differs when used in various situations. The declaration of such a structure looks like this:
**`struct`** *`identifier`* **`{`** *`set-of-declarations`* *`type`* *`array-name`* **`[]; };`**
Unsized arrays can appear only as the last member of a structure. Structures containing unsized array declarations can be nested within other structures as long as no further members are declared in any enclosing structures. Arrays of such structures aren't allowed. The **`sizeof`** operator, when applied to a variable of this type or to the type itself, assumes 0 for the s

**Contoh Kode / Perintah CLI:**

```
struct employee   /* Defines a structure variable named temp */
{
    char name[20];
    int id;
    long class;
} temp;

// ----------------------------------------
```

```
struct employee student, faculty, staff;

// ---------------------------------------

struct           /* Defines an anonymous struct and a */
{                /* structure variable named complex  */
    float x, y;
} complex;
```

**[c_015] C - Structures & Designated Initializers**

## Structure Declarations (struct, typedef struct, Member access, Designated Initializers .x = 1): See also

*Sumber: ht*

## See also
Declarators and Variable Declarations

**[c_016] C - Unions & Bit Fields**

## Union Declarations (Memory sharing, Tagged unions, Bit fields): Overview

*Sumber: ht*

# `union` Declarations
A "union declaration" specifies a set of variable values and, optionally, a tag naming the union. The variable values are called "members" of the union and can have different types. Unions are similar to "variant records" in other languages.

**[c_017] C - Unions & Bit Fields**

## Union Declarations (Memory sharing, Tagged unions, Bit fields): Syntax

*Sumber: ht*

## Syntax
*`struct-or-union-specifier`*:\
 *`struct-or-union`* *`identifier`*opt **`{`** *`struct-declaration-list`* **`}`**\
 *`struct-or-union`* *`identifier`*
*`struct-or-union`*:\
 **`struct`**\
 **`union`**
*`struct-declaration-list`*:\
 *`struct-declaration`*\
 *`struct-declaration-list`* *`struct-declaration`*
The union content is defined to be
*`struct-declaration`*:\
 *`specifier-qualifier-list`* *`struct-declarator-list`* **`;`**
*`specifier-qualifier-list`*:\
 *`type-specifier`* *`specifier-qualifier-list`*opt \
 *`type-qualifier`* *`specifier-qualifier-list`*opt
*`struct-declarator-list`*:\
 *`struct-declarator`*\
 *`struct-declarator-list`* **`,`** *`struct-declarator`*
A variable with **`union`** type stores one of the values defined by that type. The same rules govern structure and union declarations. Unions can also have bit fields.
Members of unions can't have an incomplete type, type **`void`**, or function type. Therefore members can't be an instance of the union but can be pointers to the union type being declared.
A union type declaration is a template only. Memory isn't reserved until the variable is declared.

> [!NOTE]
> If a union of two types is declared and one value is stored, but the union is accessed with the other type, the results are unreliable. For example, a union of **`float`** and **`int`** is declared. A **`float`** value is stored, but the program later accesses the value as an **`int`**. In such a situation, the value would depend on the internal storage of **`float`** values. The integer value would not be reliable.

**[c_018] C - Unions & Bit Fields**

## Union Declarations (Memory sharing, Tagged unions, Bit fields): Examples

*Sumber: ht*

## Examples
The following are examples of unions:
This example defines a union variable with `sign` type and declares a variable named `number` that has two members: `svar`, a signed integer, and `uvar`, an unsigned integer. This declaration allows the current value of `number` to be stored as either a signed or an unsigned value. The tag associated with this union type is `sign`.
The `screen` array contains 2,000 elements. Each element of the array is an individual union with two members: `window1` and `screenval`. The `window1` member is a structure with two bit-field members, `icon` and `color`. The `screenval` member is an **`int`**. At any given time, each union element holds either the **`int`** represented by `screenval` or the structure represented by `window1`.
**Microsoft Specific**
Nested unions can be declared anonymously when they're members of another structure or union. Here's an example of a nameless union:
Unions are often nested within a structure that includes a field giving the type of data contained in the union at any particular time. Here's an example of a declaration for such a union:
See Structure and Union Members for information about referencing unions.
**END Microsoft Specific**

**Contoh Kode / Perintah CLI:**

```
union sign   /* A definition and a declaration */
{
    int svar;
    unsigned uvar;
} number;

// ----------------------------------------

union               /* Defines a two-dimensional */
{                   /*  array named screen */
    struct
    {
      unsigned int icon : 8;
      unsigned color : 4;
    } window1;
    int screenval;
} screen[25][80];

// ----------------------------------------

struct str
{
    int a, b;
    union            /* Unnamed union */
    {
      char c[4];
      long l;
      float f;
   };
   char c_array[10];
```

```
} my_str;
.
.
.
my_str.l == 0L;  /* A reference to a field in the my_str union */
```

**[c_019] C - Unions & Bit Fields**

## Union Declarations (Memory sharing, Tagged unions, Bit fields): See also

*Sumber: ht*

## See also
Declarators and Variable Declarations

**[c_020] C - Functions & Variadic Arguments**

## Function Definitions (Parameters, static functions, inline, Variadic arguments va_list): Overview

*Sumber: ht*

# C Function Definitions
A function definition specifies the name of the function, the types and number of parameters it expects to receive, and its return type. A function definition also includes a function body with the declarations of its local variables, and the statements that determine what the function does.

**[c_021] C - Functions & Variadic Arguments**

## Function Definitions (Parameters, static functions, inline, Variadic arguments va_list): Syntax

*Sumber: ht*

## Syntax
*`translation-unit`*:\
 *`external-declaration`* \
 *`translation-unit`* *`external-declaration`*
*`external-declaration`*: /* Allowed only at external (file) scope */\
 *`function-definition`*\
 *`declaration`*
*`function-definition`*:\
 *`declaration-specifiers`*opt *`attribute-seq`*opt *`declarator`* *`declaration-list`*opt *`compound-statement`*
/* *`attribute-seq`* is Microsoft-specific */
Prototype parameters are:
*`declaration-specifiers`*:\
 *`storage-class-specifier`* *`declaration-specifiers`*opt \
 *`type-specifier`* *`declaration-specifiers`*opt\
 *`type-qualifier`* *`declaration-specifiers`*opt
*`declaration-list`*:\
 *`declaration`*\
 *`declaration-list`* *`declaration`*
*`declarator`*:\
 *`pointer`*opt *`direct-declarator`*
*`direct-declarator`*: /* A function declarator */\
 *`direct-declarator`* **`(`** *`parameter-type-list`* **`)`** /* New-style declarator */\
 *`direct-declarator`* **`(`** *`identifier-list`*opt **`)`** /* Obsolete-style declarator */
The parameter list in a definition uses this syntax:
*`parameter-type-list`*: /* The parameter list */\

 *`parameter-list`* \
 *`parameter-list`* **`, ...`**
*`parameter-list`*:\
 *`parameter-declaration`*\
 *`parameter-list`* **`,`** *`parameter-declaration`*
*`parameter-declaration`*:\
 *`declaration-specifiers`* *`declarator`*\
 *`declaration-specifiers`* *`abstract-declarator`*opt
The parameter list in an old-style function definition uses this syntax:
*`identifier-list`*: /* Used in obsolete-style function definitions and declarations */\
 *`identifier`*\
 *`identifier-list`* **`,`** *`identifier`*
The syntax for the function body is:
*`compound-statement`*:\
 

**[c_022] C - Control Flow - Selection (if & switch)**

## Selection Statements (switch, case, break, fallthrough, if-else): Overview

*Sumber: ht*

# `switch` Statement (C)
The **`switch`** and **`case`** statements help control complex conditional and branching operations. The **`switch`** statement transfers control to a statement within its body.

**[c_023] C - Control Flow - Selection (if & switch)**

## Selection Statements (switch, case, break, fallthrough, if-else): Syntax

*Sumber: ht*

## Syntax
*`selection-statement`*:\
  **`switch (`** *`expression`* **`)`** *`statement`*
*`labeled-statement`*:\
  **`case`** *`constant-expression`* **`:`** *`statement`*\
  **`default`** **`:`** *`statement`*

**[c_024] C - Control Flow - Selection (if & switch)**

## Selection Statements (switch, case, break, fallthrough, if-else): Remarks

*Sumber: ht*

## Remarks
A **`switch`** statement causes control to transfer to one *`labeled-statement`* in its statement body, depending on the value of *`expression`*.
The values of *`expression`* and each *`constant-expression`* must have an integral type. A *`constant-expression`* must have an unambiguous constant integral value at compile time.
Control passes to the **`case`** statement whose *`constant-expression`* value matches the value of *`expression`*. The **`switch`** statement can include any number of **`case`** instances. However, no two *`constant-expression`* values within the same **`switch`** statement can have the same value. Execution of the **`switch`** statement body begins at the first statement in or after the matching *`labeled-statement`*. Execution proceeds until the end of the body, or until a **`break`** statement transfers control out of the body.
Use of the **`switch`** statement usually looks something like this:
You can use the **`break`** statement to end processing of a particular labeled statement within the **`switch`** statement. It branches to the end of the **`switch`** statement. Without **`break`**, the program continues to the next labeled statement, executing the statements until a **`break`** or the end of the statement is reached. This continuation

may be desirable in some situations.

The **`default`** statement is executed if no **`case`** *`constant-expression`* value is equal to the value of *`expression`*. If there's no **`default`** statement, and no **`case`** match is found, none of the statements in the **`switch`** body get executed. There can be at most one **`default`** statement. The **`default`** statement doesn't have to come at the end. It may appear anywhere in the body of the **`switch`** statement. A **`case`** or **`d

**Contoh Kode / Perintah CLI:**

```
switch ( expression )
{
    // declarations
    // . . .
    case constant_expression:
        // statements executed if the expression equals the
        // value of this constant_expression
        break;
    default:
        // statements executed if expression does not equal
        // any case constant_expression
}

// ----------------------------------------

switch( c )
{
    case 'A':
        capital_a++;
    case 'a':
        letter_a++;
    default :
        total++;
}

// ----------------------------------------

switch( i )
{
    case -1:
        n++;
        break;
    case 0 :
        z++;
        break;
    case 1 :
        p++;
        break;
}
```

**[c_025] C - Control Flow - Selection (if & switch)**

## Selection Statements (switch, case, break, fallthrough, if-else): Microsoft-specific

*Sumber: ht*

### Microsoft-specific

Microsoft C doesn't limit the number of **`case`** values in a **`switch`** statement. The number is limited only by the available memory. ANSI C requires at least 257 **`case`** labels be allowed in a **`switch`** statement.

The default for Microsoft C is that the Microsoft extensions are enabled. Use the /Za compiler option to disable these extensions.

**[c_026] C - Control Flow - Selection (if & switch)**

## Selection Statements (switch, case, break, fallthrough, if-else): See also

*Sumber: ht*

## See also
switch Statement (C++)

**[c_027] C - Control Flow - Iteration (for & while)**

## Iteration Statements (for loops with C99/C11 loop variable declarations, while, do-while): Overview

*Sumber: ht*

# `for` Statement (C)
The **`for`** statement lets you repeat a statement or compound statement a specified number of times. The body of a **`for`** statement is executed zero or more times until an optional condition becomes false. You can use optional expressions within the **`for`** statement to initialize and change values during the **`for`** statement's execution.

**[c_028] C - Control Flow - Iteration (for & while)**

## Iteration Statements (for loops with C99/C11 loop variable declarations, while, do-while): Syntax

*Sumber: ht*

## Syntax
*`iteration-statement`*:\
 **`for`** **`(`** *`init-expression`*opt **`;`** *`cond-expression`*opt **`;`** *`loop-expression`*opt **`)`** *`statement`*
Execution of a **`for`** statement proceeds as follows:
1. The *`init-expression`*, if any, is evaluated. It specifies the initialization for the loop. There's no restriction on the type of *`init-expression`*.
1. The *`cond-expression`*, if any, is evaluated. This expression must have arithmetic or pointer type. It's evaluated before each iteration. Three results are possible:
- If *`cond-expression`* is **`true`** (nonzero), *`statement`* is executed; then *`loop-expression`*, if any, is evaluated. The *`loop-expression`* is evaluated after each iteration. There's no restriction on its type. Side effects will execute in order. The process then begins again with the evaluation of *`cond-expression`*.
- If *`cond-expression`* is omitted, *`cond-expression`* is considered true, and execution proceeds exactly as described in the previous paragraph. A **`for`** statement without a *`cond-expression`* argument terminates only when a **`break`** or **`return`** statement within the statement body is executed, or when a **`goto`** (to a labeled statement outside the **`for`** statement body) is executed.
- If *`cond-expression`* is **`false`** (0), execution of the **`for`** statement terminates and control passes to the next statement in the program.
A **`for`** statement also terminates when a **`break`**, **`goto`**, or **`return`** statement within the statement body is executed. A **`continue`** statement in a **`for`** loop causes *`loop-expression`* to be evaluated. When a **`break`** statement is executed inside a **`for`** loop, *`loop-expression`* isn't evaluated or executed. This statement is the cust

**Contoh Kode / Perintah CLI:**
```
for( ; ; )
```

**[c_029] C - Control Flow - Iteration (for & while)**

## Iteration Statements (for loops with C99/C11 loop variable declarations, while, do-while): Example

*Sumber: ht*

## Example
This example illustrates the **`for`** statement:

**Contoh Kode / Perintah CLI:**

```
// c_for.c
int main()
{
   char* line = "H e  \tl\tlo World\0";
   int space = 0;
   int tab = 0;
   int i;
   int max = strlen(line);
   for (i = 0; i < max; i++ )
   {
      if ( line[i] == ' ' )
      {
          space++;
      }
      if ( line[i] == '\t' )
      {
          tab++;
      }
   }

   printf("Number of spaces: %i\n", space);
   printf("Number of tabs: %i\n", tab);
   return 0;
}
```

**[c_030] C - Modern Features - C11/C17/C23 Generic Selection**

## Generic Selection (_Generic keyword for type-directed macro dispatch and polymorphism): Overview

*Sumber: ht*

# Generic selection (C11)
Use the **`_Generic`** keyword to write code that selects an expression at compile time based on the type of the argument. It's similar to overloading in C++ where the type of the argument selects which function to call. In this case, the type of the argument selects which expression to evaluate.
For example, the expression `_Generic(42, int: "integer", char: "character", default: "unknown");` evaluates the type of `42` and looks for the matching type, `int`, in the list. It finds it and returns `"integer"`.

**[c_031] C - Modern Features - C11/C17/C23 Generic Selection**

## Generic Selection (_Generic keyword for type-directed macro dispatch and polymorphism): Syntax

*Sumber: ht*

## Syntax
*`generic-selection`*:\
 **`_Generic`** **(** *`assignment-expression`, `assoc-list`* **)**
*`assoc-list`*:\
 *`association`*\
 *`assoc-list`, `association`*

*`association`*:\
 *`type-name`* : *`assignment-expression`*\
 **`default`** : *`assignment-expression`*
The first *`assignment-expression`* is called the controlling expression. The type of the controlling expression is determined at compile time and matched against the *`assoc-list`* to find which expression to evaluate and return. The controlling expression isn't evaluated. For example, `_Generic(intFunc(), int: "integer", default: "error");` doesn't result in a call at runtime to `intFunc`.
When the type of the controlling expression is determined, `const`, `volatile`, and `restrict` are removed before matching against *`assoc-list`*.
Entries in the `assoc-list` that aren't chosen aren't evaluated.

**[c_032] C - Modern Features - C11/C17/C23 Generic Selection**

## Generic Selection (_Generic keyword for type-directed macro dispatch and polymorphism): Constraints

*Sumber: ht*

## Constraints
- The *`assoc-list`* can't specify the same type more than once.
- The *`assoc-list`* can't specify types that are compatible with each other, such as an enumeration and the underlying type of that enumeration.
- If a generic selection doesn't have a default, the controlling expression must have only one compatible type name in the generic association list.

**[c_033] C - Modern Features - C11/C17/C23 Generic Selection**

## Generic Selection (_Generic keyword for type-directed macro dispatch and polymorphism): Example

*Sumber: ht*

## Example
One way to use **`_Generic`** is in a macro. The \ header file uses **`_Generic`** to call the right math function depending on the type of argument. For example, the macro for `cos` maps a call with a float to `cosf`, while mapping a call with a complex double to `ccos`.
The following example shows how to write a macro that identifies the type of the argument you pass to it. It produces `"unknown"` if no entry in the *`assoc-list`* matches the controlling expression:

**Contoh Kode / Perintah CLI:**

```
// Compile with /std:c11

#include <stdio.h>

/* Get a type name string for the argument x */
#define TYPE_NAME(X) _Generic((X), \
      int: "int", \
      char: "char", \
      double: "double", \
      default: "unknown")

int main()
{
    printf("Type name: %s\n", TYPE_NAME(42.42));

    // The following would result in a compile error because
    // 42.4 is a double, doesn't match anything in the list,
    // and there is no default.
    // _Generic(42.4, int: "integer", char: "character"));
}
```

```
/* Output:
Type name: double
*/
```

[c_034] C - Modern Features - C11/C17/C23 Generic Selection

## Generic Selection (_Generic keyword for type-directed macro dispatch and polymorphism): Requirements

Sumber: ht

## Requirements
Compile with `/std:c11`.
Windows SDK 10.0.20348.0 (version 2104) or later. See Windows SDK to download the latest SDK. For instructions to install and use the SDK for C11 and C17 development, see Install C11 and C17 support in Visual Studio.

[c_035] C - Modern Features - C11/C17/C23 Generic Selection

## Generic Selection (_Generic keyword for type-directed macro dispatch and polymorphism): See also

Sumber: ht

## See also
`/std` (Specify language standard version)\
Type-generic math

[c_036] C - Dynamic Memory Allocation (Heap Management)

## Dynamic Memory Allocation (malloc, free, calloc, realloc, Heap safety & Memory Leaks): Overview

Sumber: ht

# `malloc`
Allocates memory blocks.

[c_037] C - Dynamic Memory Allocation (Heap Management)

## Dynamic Memory Allocation (malloc, free, calloc, realloc, Heap safety & Memory Leaks): Syntax

Sumber: ht

## Syntax

**Contoh Kode / Perintah CLI:**

```
void *malloc(
   size_t size
);
```

[c_038] C - Dynamic Memory Allocation (Heap Management)

## Dynamic Memory Allocation (malloc, free, calloc, realloc, Heap safety & Memory Leaks): Parameters

Sumber: ht

### Parameters
*`size`*\
Bytes to allocate.

**[c_039] C - Dynamic Memory Allocation (Heap Management)**

## Dynamic Memory Allocation (malloc, free, calloc, realloc, Heap safety & Memory Leaks): Return value

*Sumber: ht*

## Return value
**`malloc`** returns a void pointer to the allocated space, or `NULL` if there's insufficient memory available. To return a pointer to a type other than **`void`**, use a type cast on the return value. The storage space pointed to by the return value is suitably aligned for storage of any type of object that has an alignment requirement less than or equal to that of the fundamental alignment. (In Visual C++, the fundamental alignment is the alignment that's required for a **`double`**, or 8 bytes. In code that targets 64-bit platforms, it's 16 bytes.) Use `_aligned_malloc` to allocate storage for objects that have a larger alignment requirement -- for example, the SSE types `__m128` and **`__m256`**, and types that are declared by using `__declspec(align( n ))` where **`n`** is greater than 8. If *`size`* is 0, **`malloc`** allocates a zero-length item in the heap and returns a valid pointer to that item. Always check the return from **`malloc`**, even if the amount of memory requested is small.

**[c_040] C - Dynamic Memory Allocation (Heap Management)**

## Dynamic Memory Allocation (malloc, free, calloc, realloc, Heap safety & Memory Leaks): Remarks

*Sumber: ht*

## Remarks
The **`malloc`** function allocates a memory block of at least *`size`* bytes. The block may be larger than *`size`* bytes because of the space that's required for alignment and maintenance information.
**`malloc`** sets `errno` to `ENOMEM` if a memory allocation fails or if the amount of memory requested exceeds `_HEAP_MAXREQ`. For information about this and other error codes, see `errno`, `_doserrno`, `_sys_errlist`, and `_sys_nerr`.
The startup code uses **`malloc`** to allocate storage for the **`_environ`**, *`envp`*, and *`argv`* variables. The following functions and their wide-character counterparts also call **`malloc`**.
:::row:::
:::column span="":::
`calloc`\
`_exec` functions\
`fgetc`\
`_fgetchar`\
`fgets`\
`fprintf`\
`fputc`\
`_fputchar`\
`fputs`\
`fread`
:::column-end:::
:::column span="":::
`fscanf`\
`fseek`\
`fsetpos`\
`_fullpath`\
`fwrite`\
`getc`\
`getchar`\
`_getcwd`\
`_getdcwd`\

`gets`
:::column-end:::
:::column span="":::
`_getw`\
`_popen`\
`printf`\
`putc`\
`putchar`\
`_putenv`\
`puts`\
`_putw`\
`scanf`
:::column-end:::
:::column span="":::
`_searchenv`\
`setvbuf`\
`_spawn` functions\
`_strdup`\
`system`\
`_tempnam`\
`ungetc`\
`vfprintf`\
`vprintf`
:::column-end:::
:::row-end:::
The C++ `_set_new_mode` function sets the new handler mode for **`malloc`**. The new handler mode indicates whether, on failure, **`malloc`** is to call the new handler routine as set by `_set_new_handler`. By default, **`malloc`** doesn't call the new handler routine on failure to allocate memory. You can override this default behavior so that, when **`malloc`** fails to allocate memory, **`malloc`** calls the new handler routine in the same way that the **`new`** operator does when it fails for the same reason. To override the default, call `_set_new_mode(1)`

**[c_041] C - Dynamic Memory Allocation (Heap Management)**

## Dynamic Memory Allocation (malloc, free, calloc, realloc, Heap safety & Memory Leaks): Requirements

*Sumber: ht*

## Requirements
| Routine | Required header |
|---|---|
| **`malloc`** | `` and `` |
For more compatibility information, see Compatibility.

**[c_042] C - Dynamic Memory Allocation (Heap Management)**

## Dynamic Memory Allocation (malloc, free, calloc, realloc, Heap safety & Memory Leaks): Libraries

*Sumber: ht*

## Libraries
All versions of the C run-time libraries.

[c_043] C - Dynamic Memory Allocation (Heap Management)

## Dynamic Memory Allocation (malloc, free, calloc, realloc, Heap safety & Memory Leaks): Example

Sumber: ht

## Example

**Contoh Kode / Perintah CLI:**

```
// crt_malloc.c
// This program allocates memory with
// malloc, then frees the memory with free.

#include <stdlib.h>   // For _MAX_PATH definition
#include <stdio.h>
#include <malloc.h>

int main( void )
{
   char *string;

   // Allocate space for a path name
   string = malloc( _MAX_PATH );

   // In a C++ file, explicitly cast malloc's return.  For example,
   // string = (char *)malloc( _MAX_PATH );

   if( string == NULL )
      printf( "Insufficient memory available\n" );
   else
   {
      printf( "Memory space allocated for path name\n" );
      free( string );
      printf( "Memory freed\n" );
   }
}
```

[c_044] C - Dynamic Memory Allocation (Heap Management)

## Dynamic Memory Allocation (malloc, free, calloc, realloc, Heap safety & Memory Leaks): See also

Sumber: ht

## See also

Memory allocation\
`calloc`\
`free`\
`realloc`\
`_aligned_malloc`

[c_045] C - Modern Features - C23 Standard (ISO C23)

## ISO C23 Modern Language Enhancements (nullptr, auto, constexpr, bool): C23 nullptr, auto Type Inference, and constexpr

Sumber: ht

C23 adalah revisi standar bahasa C terbaru (ISO/IEC 9899:2024). Perubahan utama meliputi:
1. nullptr: C23 menambahkan konstanta 'nullptr' dari tipe 'nullptr_t' untuk menggantikan makro NULL yang ambigu.
2. auto Type Inference: Kata kunci 'auto' kini dapat digunakan untuk inferensi tipe otomatis variabel seperti di C++: auto x = 42;

3. constexpr Object Specifier: Mendukung 'constexpr' untuk variabel objek waktu kompilasi sejati.
4. bool, true, false menjadi keyword resmi C23 tanpa memerlukan header <stdbool.h>.
5. Standard Attributes: Sintaks atribut standar ganda [[nodiscard]], [[deprecated]], [[maybe_unused]], [[fallthrough]].
6. #embed Preprocessor: Perintah preprocessor baru untuk menyematkan data biner mentah secara langsung saat kompilasi.

**Contoh Kode / Perintah CLI:**

```
// Contoh Fitur Modern C23
#include <stdio.h>

// C23 Attributes & bool tanpa stdbool.h
[[nodiscard]] bool check_pointer(void* ptr) {
    return ptr != nullptr; // C23 nullptr
}

int main(void) {
    auto count = 100; // C23 auto type inference
    constexpr double PI = 3.1415926535; // C23 constexpr

    int* p = nullptr;
    if (!check_pointer(p)) {
        printf("Pointer is null\n");
    }
    return 0;
}
```